

## Part-B Lab Questions

---

Q1.

- (a) Create a 1D NumPy array with values 1 to 20.
- (b) Extract all prime numbers from it.
- (c) Compute the mean and variance of the extracted primes.

```
In [1]: import numpy as np

# 1.(a) Create 1D NumPy array with values 1 to 20
arr=np.arange(1,21)
print("Array values: ",arr)

# 1.(b) Extract prime numbers
def is_prime(num):
    if num<2:
        return False
    for i in range(2,int(num**0.5)+1):
        if num%i==0:
            return False
    return True
prime_list=[]
for x in arr:
    if is_prime(x):
        prime_list.append(x)
prime_numbers=np.array(prime_list)
print("Prime numbers are:",prime_numbers)

# 1.(c) Compute mean and variance of primes
print("Mean of the extracted primes: ",prime_numbers.mean())
print("Variance of the extracted primes:",prime_numbers.var())
```

```
Array values: [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
Prime numbers are: [ 2  3  5  7 11 13 17 19]
Mean of the extracted primes: 9.625
Variance of the extracted primes: 35.734375
```

---

Q2.

- (a) Create a 4×4 NumPy array with numbers 1 to 16.
- (b) Extract the 2×2 bottom-left sub-matrix.
- (c) Compute the determinant of the sub-matrix.

```
In [2]: # 2.(a) Create 4x4 array with numbers 1 to 16
arr=np.arange(1,17).reshape(4,4)
print("Array of dimensions 4X4: \n",arr)

# 2.(b) Extract 2x2 bottom-left sub-matrix
sub_of_matrix=arr[2,:2]
print("Bottom-Left Sub-matrix (2X2): \n",sub_of_matrix)

# 2.(c) Compute determinant
determinant=np.linalg.det(sub_of_matrix)
print("Determinant is: ",determinant)
```

Array of dimensions 4X4:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
```

Bottom-Left Sub-matrix (2X2):

```
[[ 9 10]
 [13 14]]
```

Determinant is: -4.000000000000009

---

### Q3.

- (a) Create a DataFrame with 5 students and marks in 3 subjects.
- (b) Add a column for total and average marks.
- (c) Identify the topper and print their name with average.

```
In [3]: import pandas as pd

# 3.(a) Create DataFrame with 5 students and 3 subjects
data_values={'Name':['Raj','Kaushal','Vikram','Deepti','Akansha'],'Math':[85,78,92,70,88],
df=pd.DataFrame(data_values)
print(df,"\n")

# 3.(b) Add Total and Average
df['Total']=df[['Math','Physics','Chemistry']].sum(axis=1)
df['Average']=df['Total']/3
print(df)

# 3.(c) Identify topper
max_avg=df['Average'].max()
topper=df[df['Average']==max_avg].iloc[0]
print(f"\nTopper: {topper['Name']}, Average: {topper['Average']}")
```

|   | Name    | Math | Physics | Chemistry |
|---|---------|------|---------|-----------|
| 0 | Raj     | 85   | 80      | 78        |
| 1 | Kaushal | 78   | 82      | 88        |
| 2 | Vikram  | 92   | 95      | 91        |
| 3 | Deepti  | 70   | 65      | 72        |
| 4 | Akansha | 88   | 90      | 85        |

|   | Name    | Math | Physics | Chemistry | Total | Average   |
|---|---------|------|---------|-----------|-------|-----------|
| 0 | Raj     | 85   | 80      | 78        | 243   | 81.000000 |
| 1 | Kaushal | 78   | 82      | 88        | 248   | 82.666667 |
| 2 | Vikram  | 92   | 95      | 91        | 278   | 92.666667 |
| 3 | Deepti  | 70   | 65      | 72        | 207   | 69.000000 |
| 4 | Akansha | 88   | 90      | 85        | 263   | 87.666667 |

Topper: Vikram, Average: 92.66666666666667

---

### Q4.

- (a) Simulate 1000 coin tosses using NumPy (1=Head, 0=Tail).
- (b) Count frequency of heads and tails.
- (c) Estimate probability of heads. Is it close to 0.5? Why/Why not?

```
In [4]: # 4.(a) Simulate 1000 coin tosses
toss=np.random.randint(0,2,size=1000)

# 4.(b) Count frequency
head=np.sum(toss==1)
tail=np.sum(toss==0)
print("Heads:",head," Tails:",tail)
```

```
# 4.(c) Estimate probability of heads
probability_heads=head/1000
print("The Probability of Heads: ",probability_heads)

# Value is close to 0.5 due to Law of Large numbers.
```

Heads: 477 Tails: 523  
The Probability of Heads: 0.477

Q5.

- (a) Create a DataFrame of employees with columns: ID, Name, Salary.
- (b) Add a Bonus column = 10% of Salary.
- (c) Display employees with salary above average.

```
In [5]: # Q5
# 5.(a) Create DataFrame of employees
employeeed={'ID':[101,102,103,104,105],'Name':['Raj','Kaushal','Vikram','Deepti','Akansha'],
df=pd.DataFrame(employeeed)
print(df,"\n")

# 5.(b) Add Bonus column=10% of Salary
df['Bonus']=df['Salary']*0.1
print(df,"\n")

# 5.(c) Employees with salary above average
avg_salary=df['Salary'].mean()
greater_avg=df[df['Salary']>avg_salary]
print("Above Average Salary Employees: \n",greater_avg)
```

|   | ID  | Name    | Salary |
|---|-----|---------|--------|
| 0 | 101 | Raj     | 50000  |
| 1 | 102 | Kaushal | 60000  |
| 2 | 103 | Vikram  | 55000  |
| 3 | 104 | Deepti  | 65000  |
| 4 | 105 | Akansha | 58000  |

|   | ID  | Name    | Salary | Bonus  |
|---|-----|---------|--------|--------|
| 0 | 101 | Raj     | 50000  | 5000.0 |
| 1 | 102 | Kaushal | 60000  | 6000.0 |
| 2 | 103 | Vikram  | 55000  | 5500.0 |
| 3 | 104 | Deepti  | 65000  | 6500.0 |
| 4 | 105 | Akansha | 58000  | 5800.0 |

Above Average Salary Employees:

|   | ID  | Name    | Salary | Bonus  |
|---|-----|---------|--------|--------|
| 1 | 102 | Kaushal | 60000  | 6000.0 |
| 3 | 104 | Deepti  | 65000  | 6500.0 |
| 4 | 105 | Akansha | 58000  | 5800.0 |

Q6.

- (a) Create a 3×3 NumPy array with values 1 to 9.
- (b) Find its transpose and inverse.
- (c) Verify that  $A \times A^{-1} \approx I$ .

```
In [6]: # 6.(a) Create 3x3 array with values 1 to 9
Mat_A=np.array([[1,2,3],[0,1,4],[5,6,0]])
print("Matrix A: \n",Mat_A)

# 6.(b) Find transpose and inverse
Mat_A_T=Mat_A.T
print("Transpose: \n",Mat_A_T)
```

```
# Inverse
A_inv=np.linalg.inv(Mat_A)
print("Inverse: \n",A_inv)

# 6.(c) Verify  $A \cdot A_{inv} = I$ 
Identity=np.dot(Mat_A,A_inv)
print("A*A_inv: \n",Identity)
```

```
Matrix A:
[[1 2 3]
 [0 1 4]
 [5 6 0]]
Transpose:
[[1 0 5]
 [2 1 6]
 [3 4 0]]
Inverse:
[[-24.  18.   5.]
 [ 20. -15.  -4.]
 [ -5.   4.   1.]]
A*A_inv:
[[ 1.00000000e+00 -2.66453526e-15  0.00000000e+00]
 [ 0.00000000e+00  1.00000000e+00  0.00000000e+00]
 [ 0.00000000e+00 -7.10542736e-15  1.00000000e+00]]
```

---

Q7.

- (a) Generate random daily temperatures (30 values, range 20–40°C).
- (b) Find the hottest and coldest day.
- (c) Compute mean, median, and standard deviation of temperatures.

```
In [7]: # 7.(a) Generate random daily temperatures (30 values, 20-40°C)
temperatures=np.random.randint(20,41,30)
print("Temperatures: ",temperatures)

# 7.(b) Hottest and coldest day
hottest_day=0
coldest_day=0
for i in range(1,len(temperatures)):
    if temperatures[i]>temperatures[hottest_day]:
        hottest_day=i
    if temperatures[i]<temperatures[coldest_day]:
        coldest_day=i
print(f"Hottest day: {hottest_day+1}, Temp: {temperatures[hottest_day]}")
print(f"Coldest day: {coldest_day+1}, Temp: {temperatures[coldest_day]}")

# 7.(c) Mean,median,std
print("Mean of temperatures: ",temperatures.mean())
print("Median of temperatures: ",np.median(temperatures))
print("Standard Deviation of temperatures: ",temperatures.std())
```

```
Temperatures: [27 39 33 36 22 28 30 38 22 37 31 33 25 31 28 32 22 21 26 32 37 31 28 28
 30 26 27 39 30 30]
Hottest day: 2, Temp: 39
Coldest day: 18, Temp: 21
Mean of temperatures: 29.966666666666665
Median of temperatures: 30.0
Standard Deviation of temperatures: 5.0098791291695735
```

---

Q8.

- (a) Create a Pandas Series of marks for 8 students.
- (b) Replace all marks below 40 with "Fail".
- (c) Count how many students passed.

```
In [8]: import pandas as pd
import numpy as np

# 8.(a) Create Pandas Series of marks for 8 students
marks=pd.Series([55,38,72,41,29,66,90,35])
print("Marks of students: \n",marks)

# 8.(b) Replace all marks below 40 with "Fail"
marks_replaced=marks.astype(object).copy()
for i in range(len(marks_replaced)):
    if marks_replaced[i]<40:
        marks_replaced[i]="Fail"
print("\nMarks After Replacement: \n",marks_replaced)

# 8.(c) Count how many students passed
pass_marks=(marks>=40).sum()
print("\nNumber of students passed: ",pass_marks)
```

Marks of students:

|   |    |
|---|----|
| 0 | 55 |
| 1 | 38 |
| 2 | 72 |
| 3 | 41 |
| 4 | 29 |
| 5 | 66 |
| 6 | 90 |
| 7 | 35 |

dtype: int64

Marks After Replacement:

|   |      |
|---|------|
| 0 | 55   |
| 1 | Fail |
| 2 | 72   |
| 3 | 41   |
| 4 | Fail |
| 5 | 66   |
| 6 | 90   |
| 7 | Fail |

dtype: object

Number of students passed: 5

---

Q9.

- (a) Generate 500 random integers between 1 and 6 (simulate dice rolls).
- (b) Count how many times each face appears.
- (c) Compute relative frequencies and compare with theoretical 1/6.

```
In [9]: # 9.(a) Simulate 500 dice rolls
dice_roll=np.random.randint(1,7,500)

# 9.(b) Count frequency of each face
counts=[np.count_nonzero(dice_roll==i) for i in range(1,7)]
print("Face counts: ",{i+1:counts[i] for i in range(6)})

# 9.(c) Compute relative frequencies
rel_freq=[counts[i]/500 for i in range(6)]
print("Relative frequencies: ",{i+1:rel_freq[i] for i in range(6)})
```

```
# Compare with theoretical probability  $1/6 \approx 0.1667$ 
```

Face counts: {1: 91, 2: 79, 3: 82, 4: 83, 5: 83, 6: 82}

Relative frequencies: {1: 0.182, 2: 0.158, 3: 0.164, 4: 0.166, 5: 0.166, 6: 0.164}

---

#### Q10.

- (a) Create a DataFrame with 6 products (Name, Quantity, Price).
- (b) Add a column "Total = Quantity × Price".
- (c) Find which product generated maximum sales revenue.

```
In [10]: # 10.(a) Create DataFrame with 6 products
products=pd.DataFrame({'Name':['A','B','C','D','E','F'],'Quantity':[10,5,8,12,7,9],'Price':
print(products,"\n")

# 10.(b) Add Total=Quantity*Price
products['Total']=products['Quantity']*products['Price']
print(products,"\n")

# 10.(c) Product with maximum sales revenue
max_total=products['Total'].max()
max_product=products[products['Total']==max_total]
print(f"Max Revenue Product: {max_product.iloc[0]['Name']}, Total: {max_product.iloc[0]['Total']})
```

|   | Name | Quantity | Price |
|---|------|----------|-------|
| 0 | A    | 10       | 50    |
| 1 | B    | 5        | 80    |
| 2 | C    | 8        | 60    |
| 3 | D    | 12       | 40    |
| 4 | E    | 7        | 90    |
| 5 | F    | 9        | 70    |

|   | Name | Quantity | Price | Total |
|---|------|----------|-------|-------|
| 0 | A    | 10       | 50    | 500   |
| 1 | B    | 5        | 80    | 400   |
| 2 | C    | 8        | 60    | 480   |
| 3 | D    | 12       | 40    | 480   |
| 4 | E    | 7        | 90    | 630   |
| 5 | F    | 9        | 70    | 630   |

Max Revenue Product: E, Total: 630

---

#### Q11.

- (a) Create a DataFrame with some missing values (NaN).
- (b) Fill missing values with column mean.
- (c) Drop rows where more than 1 value is missing.

```
In [11]: # 11.(a) Create DataFrame with missing values
df=pd.DataFrame({'A':[1,None,3,4],'B':[5,6,np.nan,8],'C':[9,10,11,None]})
print("Original DataFrame:\n",df)

# 11.(b) Fill missing values with column mean
df1=df.fillna(df.mean())
print("\nAfter filling NaN with column mean: \n",df1)

# 11.(c) Drop rows where more than 1 value is missing
check=df1.isnull().sum(axis=1)<=1
df2=df1[check]
print("\nAfter dropping rows with >1 missing: \n",df2)
```

Original DataFrame:

|   | A   | B   | C    |
|---|-----|-----|------|
| 0 | 1.0 | 5.0 | 9.0  |
| 1 | NaN | 6.0 | 10.0 |
| 2 | 3.0 | NaN | 11.0 |
| 3 | 4.0 | 8.0 | NaN  |

After filling NaN with column mean:

|   | A        | B        | C    |
|---|----------|----------|------|
| 0 | 1.000000 | 5.000000 | 9.0  |
| 1 | 2.666667 | 6.000000 | 10.0 |
| 2 | 3.000000 | 6.333333 | 11.0 |
| 3 | 4.000000 | 8.000000 | 10.0 |

After dropping rows with >1 missing:

|   | A        | B        | C    |
|---|----------|----------|------|
| 0 | 1.000000 | 5.000000 | 9.0  |
| 1 | 2.666667 | 6.000000 | 10.0 |
| 2 | 3.000000 | 6.333333 | 11.0 |
| 3 | 4.000000 | 8.000000 | 10.0 |

---

Q12.

- (a) Create a NumPy array with integers from 1 to 30.
- (b) Reshape it into a 5×6 matrix.
- (c) Extract all even numbers from the matrix and compute their average.

```
In [12]: # 12.(a) Create NumPy array 1 to 30
arr=np.arange(1,31)

# 12.(b) Reshape into 5x6 matrix
matrix=arr.reshape(5,6)
print("5x6 Matrix:\n",matrix)

# 12.(c) Extract even numbers and compute average
evens=matrix[matrix%2==0]
print("\nEven numbers in the matrix: ",evens)
print("Average of even numbers from the matrix: ",evens.mean())
```

5x6 Matrix:

```
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]
 [25 26 27 28 29 30]]
```

Even numbers in the matrix: [ 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30]

Average of even numbers from the matrix: 16.0

---

Q13.

- (a) Create a DataFrame of 6 students with columns: Name, Age, Marks.
- (b) Select only students who scored above the overall average marks.
- (c) Display names of students younger than 20 whose marks are above 60.

```
In [13]: # 13.(a) Create DataFrame of 6 students
students=pd.DataFrame({'Name':['Anurag','Amaan','Anusha','Muskaan','Anuradha','Roshni'],'Age':
print(students,"\n")

# 13.(b) Students above overall average marks
avg_marks=students['Marks'].mean()
above_avg_students=students[students['Marks']>avg_marks]
print("Students above average marks: \n",above_avg_students)
```

```
# 13.(c) Names of students younger than 20 whose marks>60
younger_name=students[(students['Age']<20)&(students['Marks']>60)]
print("\nStudents <20 with Marks>60:")
print(younger_name['Name'])
```

|   | Name     | Age | Marks |
|---|----------|-----|-------|
| 0 | Anurag   | 19  | 65    |
| 1 | Amaan    | 21  | 55    |
| 2 | Anusha   | 18  | 70    |
| 3 | Muskaan  | 20  | 45    |
| 4 | Anuradha | 22  | 80    |
| 5 | Roshni   | 17  | 60    |

Students above average marks:

|   | Name     | Age | Marks |
|---|----------|-----|-------|
| 0 | Anurag   | 19  | 65    |
| 2 | Anusha   | 18  | 70    |
| 4 | Anuradha | 22  | 80    |

Students <20 with Marks>60:

```
0    Anurag
2    Anusha
Name: Name, dtype: object
```

#### Q14.

- (a) Create a Pandas DataFrame with 5 employees having columns: Name, Tasks\_Completed, Hours\_Worked.
- (b) Add a new column Efficiency = Tasks\_Completed / Hours\_Worked.
- (c) Identify the employee with the highest efficiency and print their name and value.

```
In [14]: # 14.(a) Create DataFrame with 5 employees
employees=pd.DataFrame({'Name':['Raj', 'Kaushal', 'Vikram', 'Deepti', 'Akansha'], 'Tasks_Completed': [50, 60, 45, 70, 55], 'Hours_Worked': [40, 50, 35, 60, 45]})
print(employees, "\n")

# 14.(b) Add Efficiency = Tasks_Completed/Hours_Worked
employees['Efficiency']=employees['Tasks_Completed']/employees['Hours_Worked']
print(employees)

# 14.(c) Employee with highest efficiency
max_eff=employees['Efficiency'].max()
best_emp=employees[employees['Efficiency']==max_eff].iloc[0]
print(f"\nHighest Efficiency: {best_emp['Name']}, Value: {best_emp['Efficiency']}")
```

|   | Name    | Tasks_Completed | Hours_Worked |
|---|---------|-----------------|--------------|
| 0 | Raj     | 50              | 40           |
| 1 | Kaushal | 60              | 50           |
| 2 | Vikram  | 45              | 35           |
| 3 | Deepti  | 70              | 60           |
| 4 | Akansha | 55              | 45           |

|   | Name    | Tasks_Completed | Hours_Worked | Efficiency |
|---|---------|-----------------|--------------|------------|
| 0 | Raj     | 50              | 40           | 1.250000   |
| 1 | Kaushal | 60              | 50           | 1.200000   |
| 2 | Vikram  | 45              | 35           | 1.285714   |
| 3 | Deepti  | 70              | 60           | 1.166667   |
| 4 | Akansha | 55              | 45           | 1.222222   |

Highest Efficiency: Vikram, Value: 1.2857142857142858